

LAB 1 결과보고서

조합논리회로의 FPGA 구현과 검증

과목	전기전자컴퓨터설계실험 2
이름	김태이
학번	2023440158
작성일	2026-09-20
대상 소자	AMD Spartan-7 xc7s75fgga484-1

저장소

github.com/tae2yi/UOS_ece2/tree/master/lab1

설계 기준 커밋

902a9c4c87ccc992ccade362fe7a69fed05b4e79

요약 10 종의 조합논리회로를 Verilog 로 구현하고, VS Code 기반 자기검사 시뮬레이션과 Vivado Behavioral Simulation 을 통해 논리 기능을 검증하였다. 모든 설계에서 비트스트림을 생성했으며, 조교 선정 실험인 Lab 07 DEMUX 와 Lab 08 Encoder 를 FPGA 보드에 기록하여 입력 조건과 LED 출력을 확인하였다. 시뮬레이션과 보드 관찰 결과는 설계식 및 기대값과 일치하였다.

1 실험 목적

본 실험의 목적은 논리 게이트, 산술회로, 비교기, 선택 회로, 부호화·복호화 회로와 7 세그먼트 디코더를 FPGA 에 구현하고 설계부터 실제 장치 동작까지 단계적으로 검증하는 것이다. 각 회로는 RTL 과 자기검사 테스트벤치를 작성한 뒤 VS Code 환경에서 전 입력 조합을 확인하고, 같은 소스와 테스트벤치를 Vivado 에서 다시 실행하여 파형을 비교하였다. 이후 합성·구현·비트스트림 생성을 수행하고, 조교가 지정한 Lab 07 과 Lab 08 을 보드에서 관찰하였다 [1, pp. 32, 34], [2, p. 10].

- RTL 의 실제 출력과 테스트벤치 기대값을 비교하여 조합논리 기능을 검증한다.
- XDC 핀 배정, 합성·배치·배선 결과와 비트스트림 생성을 확인한다.
- 선정 실험의 입력 조건과 실제 LED 출력을 사진으로 기록하고 예상값과 비교한다.

2 설계 및 검증 방법

2.1 대상 회로와 검증 범위

Lab	회로	입출력	검사 수
01	AND OR XOR	$a\ b \rightarrow x\ y\ z$	4
02	Full Adder	$a\ b\ cin \rightarrow cout\ s$	8
03	4 bit Adder	$a\ b \rightarrow cout\ s$	256
04	4 bit Subtractor	$a\ b \rightarrow bor\ d$	256
05	4 bit Comparator	$a\ b \rightarrow gt\ eq\ lt$	256
06	4 to 1 MUX	$i\ s \rightarrow z$	64
07	1 to 8 DEMUX	$i\ s \rightarrow o$	16
08	8 to 3 Encoder	$one\ hot\ i \rightarrow a$	256
09	3 to 8 Decoder	$a\ b\ c \rightarrow o$	8
10	Hex 7 Segment	$bcd \rightarrow seg\ data$	16

자기검사 테스트벤치는 입력을 순차적으로 인가하고 실제 출력이 expected 와 다른 경우 \$fatal 로 중단한다. 모든 입력을 검사한 뒤 LAB1_PASS 와 검사 수를 출력하고 \$finish 로 종료한다. VS Code 와 Vivado 는 동일한 RTL 과 테스트벤치를 사용하므로 파형의 입력·출력·시간 순서를 직접 비교할 수 있다 [2, pp. 15-20, 35-36].

2.2 FPGA 구현 절차

Vivado 2026.1 에서 대상 소자 xc7s75fgga484-1 을 선택하고 RTL 을 Design Sources, 테스트벤치를

Simulation Sources, XDC 를 Constraints 에 추가하였다. XDC 의 PACKAGE_PIN 과 IOSTANDARD 는 설계 포트를 물리 핀 및 LVCMOS33 규격에 연결하며 [3], 구현 단계는 논리 최적화, 배치, 배선과 비트스트림 생성 순으로 진행된다 [4].

- Simulation Top 은 tb 모듈, 합성 Top 은 DUT 모듈로 분리하였다.
- Behavioral Simulation 에서 실제 출력과 expected 가 같은지 확인하였다.
- Synthesis, Implementation, Generate Bitstream 완료 후 활용률·타이밍·DRC 리포트를 확인하였다.

2.3 제출 소스

전체 RTL, 테스트벤치와 XDC 는 다음 저장소에 있다. 기준 브랜치는 master 이며 본 보고서의 설계 분석은 커밋

902a9c4c87ccc992ccade362fe7a69fed05b4e79 을 기준으로 한다. [GitHub 저장소](#)

3 핵심 회로 구현

10 개 설계는 모두 현재 입력만으로 출력이 결정되는 조합논리회로이다. 단순 논리식은 연속 할당으로 기술하고, 입력 패턴별 상수가 필요한 Encoder 와 7 Segment 는 조합 always 블록과 case 문으로 구현하였다. 전체 코드는 저장소에 두고 보고서에는 회로 구조를 결정하는 핵심 부분만 제시한다.

3.1 논리 게이트와 가산기

```
assign x = a & b;
assign y = a | b;
assign z = a ^ b;

half_adder first(a,b,partial_sum,carry_ab);
half_adder second(partial_sum,cin,s,carry_cin);
assign cout = carry_ab | carry_cin;

assign {cout,s} = {1'b0,a} + {1'b0,b};
```

Lab 02 는 두 개의 반가산기를 직렬로 연결하고 두 carry 를 OR 하여 전가산기를 구성한다. Lab 03 은 4 비트 합의 상위 한 비트를 cout 으로 분리해 0 부터 30 까지의 결과를 5 비트로 보존한다.

3.2 산술 비교와 선택 회로

```
assign d = a - b;
assign bor = a < b;
assign o = {a > b, a == b, a < b};
assign z = i[3-s];
assign o = i ? (8'b100000000 >> s) : 8'b0;
assign o = 8'b00000001 << {a,b,c};
```

Comparator 출력은 {gt, eq, lt} 중 한 비트만 1 인 one hot 구조다. MUX 와 DEMUX 는 교안의 비트 순서를

유지하여 선택값 0 이 각각 i[3]과 o[7]에 대응하고, Decoder 는 입력값을 왼쪽 시프트 양으로 사용한다.

3.3 Encoder 와 7 Segment

```
case (i)
  8'h80: a=0; 8'h40: a=1; 8'h20: a=2; 8'h10: a=3;
  8'h08: a=4; 8'h04: a=5; 8'h02: a=6; 8'h01: a=7;
  default: a=0;
endcase

case (bcd)
  0: seg_data=8'hfc; 1: seg_data=8'h60;
  8: seg_data=8'hfe; 15: seg_data=8'h8e;
endcase
```

Encoder 는 one hot 입력만 유효하며 0 개 또는 여러 입력이 동시에 1 이면 000 을 출력한다. 7 Segment 출력 순서는 {a,b,c,d,e,f,g,dp}이고 active high 이며, dp 는 끄고 0 부터 F 까지 서로 다른 패턴을 출력한다.

4 실험 결과

4.1 구현 결과

표 2. 구현 자원

Lab	Top	LUT	IOB
01	logic_gate	2	5
02	full_adder	1	5
03	adder_4bit	4	13
04	sub_4bit	4	13
05	compare_4	5	11
06	mux_4x1	1	7
07	demux_1x8	4	12
08	encoder8x3	5	11
09	decoder3x8	4	11
10	seg_decoder	4	12

표 3. 가장 입출력 경로

Lab	경로	지연 ns
01	b → x	8.603
02	b → cout	8.789
03	a[0] → cout	12.109
04	a[0] → bor	12.330
05	a[0] → o[1]	11.309
06	s[0] → z	9.538
07	s[1] → o[7]	10.069
08	i[0] → a[2]	10.479
09	b → o[0]	9.390
10	bcd[0] → seg_data[4]	10.160

10 개 프로젝트에서 합성·구현과 비트스트림 생성이 모두 완료되었다. IOB 수는 각 top 의 포트 비트 수와 일치했으며, Slice LUT 사용량은 1~5 개였다. 7 Series FPGA 의 LUT 는 여러 입력의 조합 함수를 구현하며 동일 입력을 공유하는 작은 함수는 한 LUT 자원을 공유할 수 있다 [5].

4.2 Lab 01 AND OR XOR Logic Gates

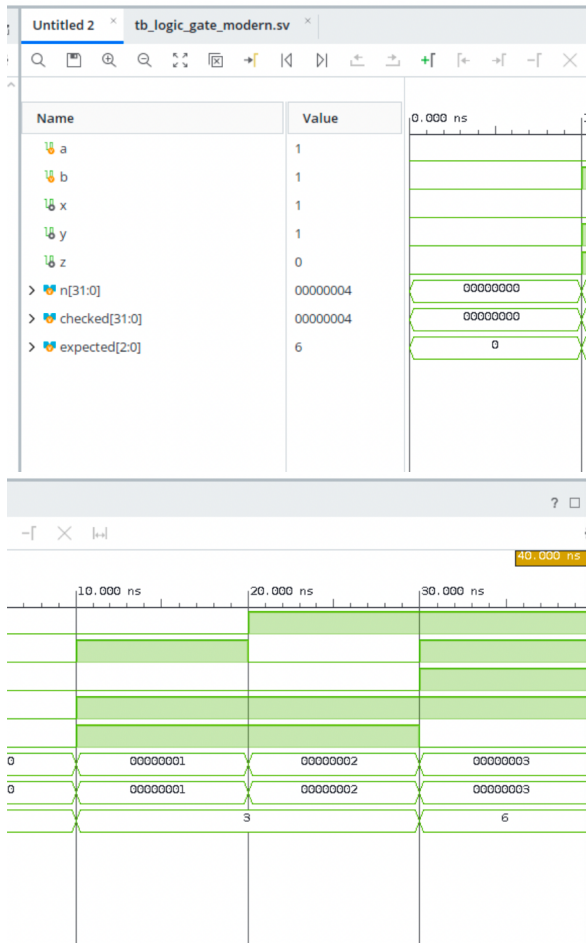


그림 1. Lab 01 Vivado Behavioral Simulation. 입력 ab=00, 01, 10, 11 에서 AND OR XOR 출력이 진리표와 일치한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

입력 a와 b의 네 조합에서 $x=a \& b$, $y=a|b$, $z=a \wedge b$ 가 각각 expected와 일치하였다. 특히 $ab=11$ 에서 x와 y는 1, z는 0이 되어 AND와 XOR의 차이가 확인되었다. VS Code 자기검사는 4개 조건을 모두 통과하였다.

4.3 Lab 02 Full Adder

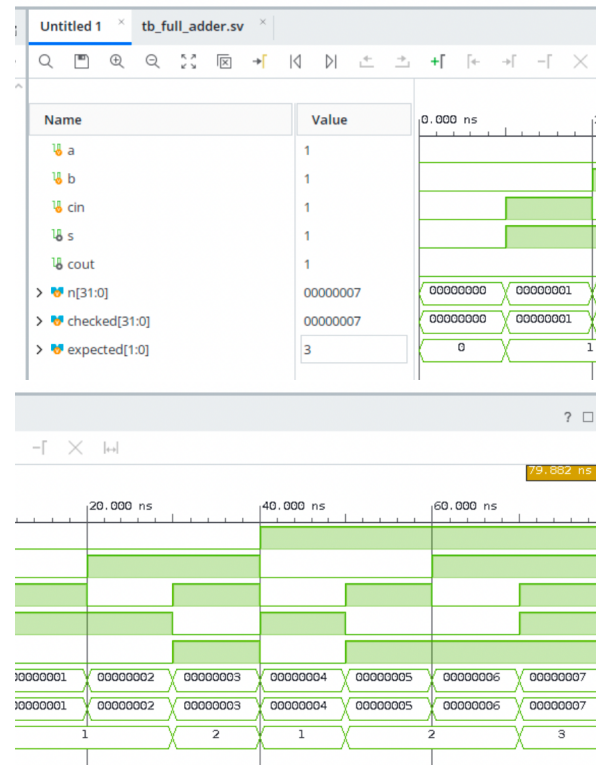


그림 2. Lab 02 전가산기 파형. abc를 000부터 111까지 변화시켰으며 {cout,s}가 세 입력의 합과 일치한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

세 입력을 000부터 111까지 변화시켰을 때 {cout,s}는 0, 1, 1, 2, 1, 2, 2, 3의 순서로 나타났다. 예를 들어 111의 합은 3이므로 $cout=1$, $s=1$ 이다. 8개 조건이 모두 통과하였다.

4.4 Lab 03 4 bit Adder

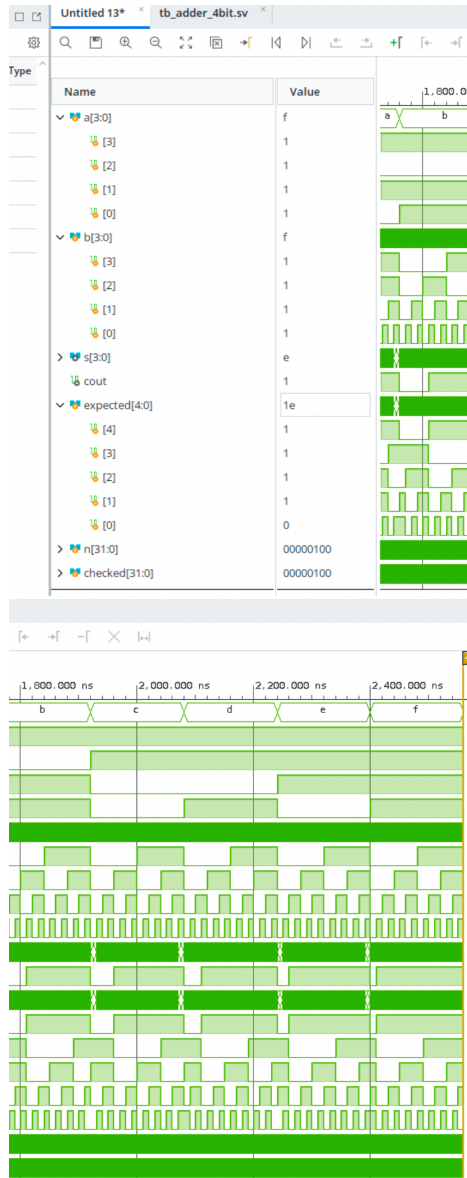


그림 3. Lab 03 4 비트 가산기 파형. 256 개 입력 조합을 2560 ns 까지 실행했으며 마지막 F+F 에서 expected=1E, cout=1, s=E 이다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

a 와 b 의 256 개 조합을 모두 실행하여 checked=256 을 확인하였다. 마지막 a=F, b=F 에서는 합 30 이 5 비트 11110 으로 나타나 expected[4]=cout=1 이고 expected[3:0]=s=1110 이었다. VS Code 와 Vivado 결과가 일치하였다.

4.5 Lab 04 4 bit Subtractor

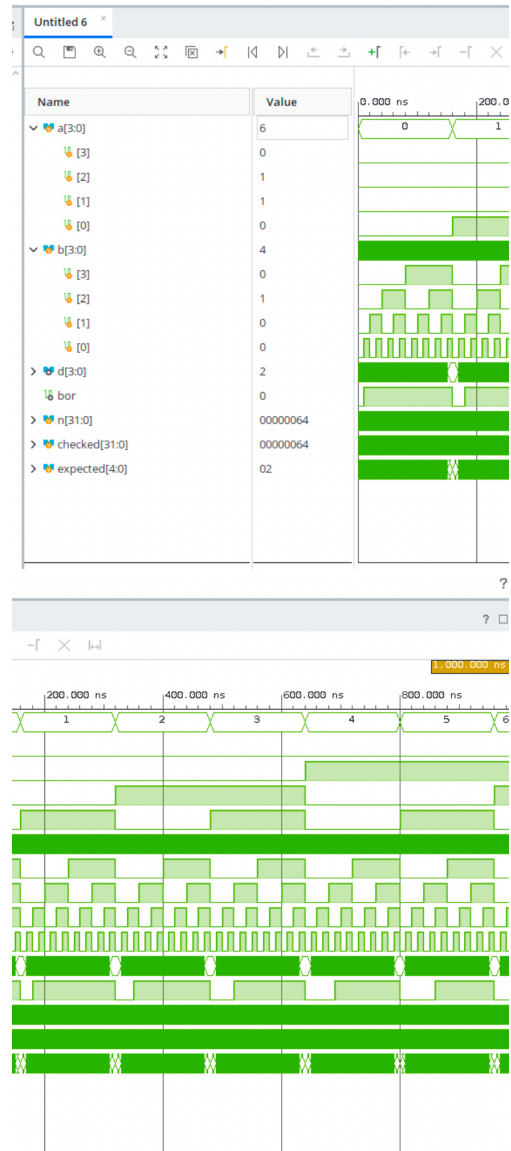


그림 4. Lab 04 4 비트 감산기 Vivado 캡처 구간. d 와 bor 가 expected의 하위 4 비트와 상위 비트에 대응한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

캡처된 구간에서 $d = a - b$ 의 하위 4 비트와 $bor = (a < b)$ 가 expected 와 동일하게 변화하였다. $a < b$ 이면 $bor = 1$ 이고 d 는 2 의 보수 형태로 나타난다. 전체 256 개 조합은 VS Code 자기검사 로그에서 PASS 를 확인하였다.

4.6 Lab 05 4 bit Comparator



그림 5. Lab 05 4 비트 비교기 Vivado 캡처 구간. $o=[a>b, a=b, a<b]$ 가 expected 와 일치한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

출력 o는 100, 010, 001 중 하나만 나타나는 one hot 조건을 유지하였다. 캡처 구간에서 실제 o 와 expected 가 일치했고, 전체 256 개 조합은 VS Code 자기검사에서 통과하였다.

4.7 Lab 06 4 to 1 Multiplexer



그림 6. Lab 06 4 대 1 MUX 파형. 모든 i와 s 조합에서 z 가 선택된 i[s]와 일치한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

입력 i의 16 개 패턴과 선택 s의 4 개 조합, 총 64 개 조건을 검사하였다. $s=00,01,10,11$ 은 각각 i[3], i[2], i[1], i[0]을 선택하며 z 와 expected 가 전 구간에서 일치하였다.

4.8 Lab 07 1 to 8 Demultiplexer



그림 7. Lab 07 1 대 8 DEMUX 파형. i=0 이면 o=00 이고 i=1 이면 s 에 따라 80 부터 01 까지 한 비트가 이동한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

i=0 에서는 선택값과 무관하게 모든 출력이 0 이었다. i=1 에서는 s=000 부터 111 까지 o 가 80,40,20,10,08,04,02,01 로 이동하여 입력 하나가 선택된 출력 하나로만 전달됨을 확인하였다. 16 개 조건이 통과하였다.

4.9 Lab 08 8 to 3 Encoder

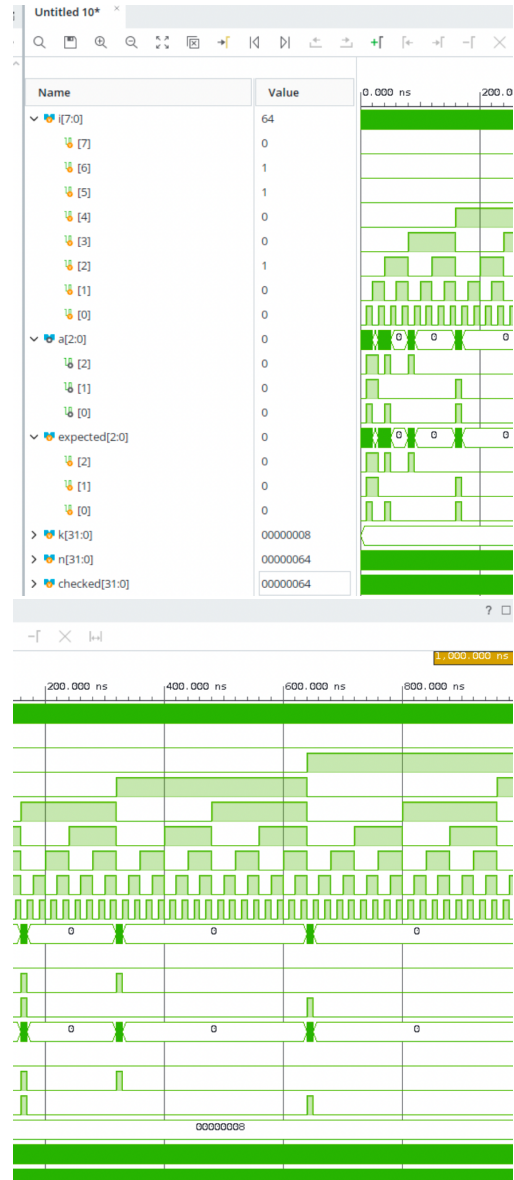


그림 8. Lab 08 8 대 3 Encoder Vivado 캡처 구간. one hot 입력은 위치값으로 부호화되고 0 또는 multi hot 입력은 000 이 된다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

유효한 one hot 입력 80,40,20,10,08,04,02,01 은 각각 0 부터 7 로 부호화된다. 정의하지 않은 0 및 multi hot 입력은 default 분기에서 000 을 출력한다. 캡처 구간의 실제값과 expected 가 일치했고 전체 256 개 입력은 VS Code 자기검사에서 통과하였다.

4.10 Lab 09 3 to 8 Decoder

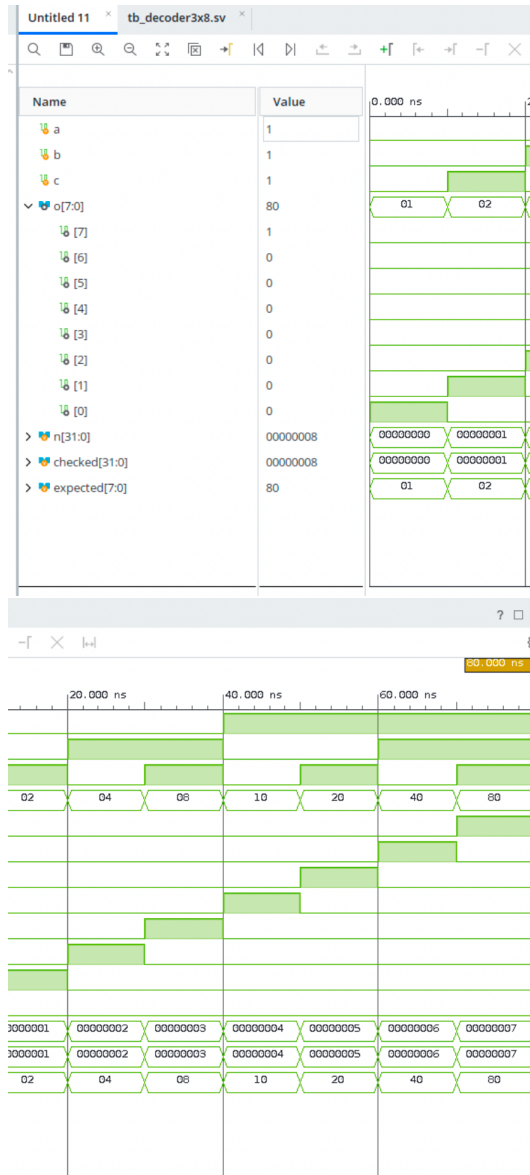


그림 9. Lab 09 3 대 8 Decoder 파형. abc=000 부터 111 까지 o가 01,02,04,08,10,20,40,80 으로 이동한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

입력 abc 를 000 부터 111 까지 증가시키면 출력의 1 이 o[0]에서 o[7]까지 이동한다. 매 구간에서 출력은 정확히 한 비트만 1 이었고 expected 와 일치하였다. 8 개 주소가 모두 통과하였다.

4.11 Lab 10 Hexadecimal 7 Segment Decoder



그림 10. Lab 10 16 진수 7 세그먼트 디코더 파형. 입력 0 부터 F 까지 seg_data 가 정의된 패턴으로 변화한다. 위쪽은 신호명과 전반 구간, 아래쪽은 이어지는 파형 구간이다.

4 비트 입력 0 부터 F 까지의 16 개 조건에서 seg_data 와 expected 가 일치하였다. 출력 비트는 {a,b,c,d,e,f,g,dp} 순서의 active high 이며 dp 는 모든 조건에서 0 이다. 입력 A 부터 F 도 서로 다른 문자 패턴으로 표현되므로 엄밀히는 BCD 가 아니라 16 진수 디코더이다.

4.12 선정 실험의 보드 동작

OT 의 보드 기록 기준에 따라 조교 선정 실험인 Lab 07 DEMUX와 Lab 08 Encoder 만 제시하였다. 각 실험에서 여러 입력 조건을 촬영했으므로 대표 사진 한 장으로 축약하지 않고, Vivado 파형과 RTL 기대값, XDC 의 핀 대응, 실제 LED 관찰을 조건별로 연결하였다 [1, p. 34].

4.12.1 Lab 07 1 to 8 DEMUX

DEMUX 는 하나의 데이터 입력 i 를 선택 신호 s 가 지정한 한 출력으로 전달하는 1 대 8 분배 회로이다. RTL 의 $o = i ? (8'b10000000 >> s) : 8'b0$ 식에서 $i=0$ 이면 모든 출력이 0 이고, $i=1$ 이면 정확히 한 비트만 1 인 one hot 출력이 생성된다. 이 설계의 비트 순서는 $s=000$ 에서 $o[7]$, $s=111$ 에서 $o[0]$ 을 선택하므로, 주소 000 이 $o[0]$ 을 선택하는 Lab 09 Decoder 와 반대이다.

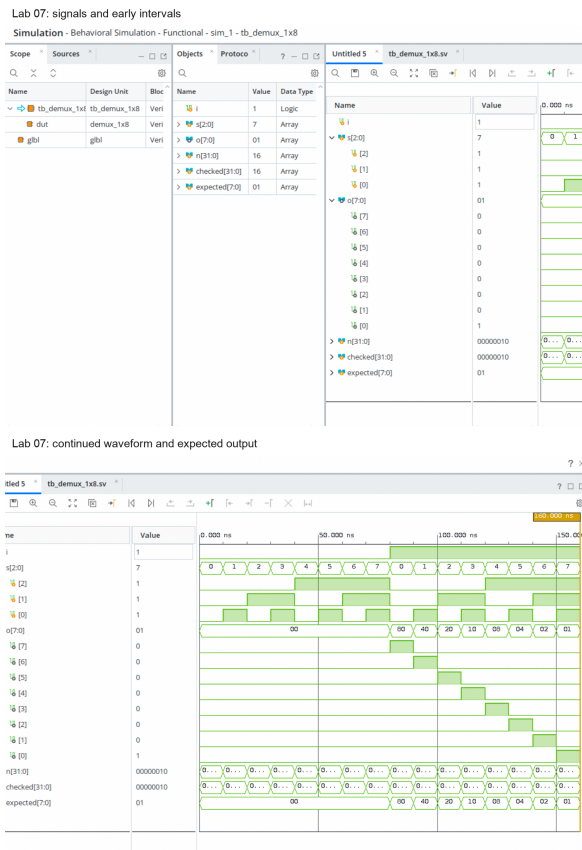


그림 11. Lab 07 Vivado 파형의 재구성. $i=0$ 인 앞의 8 개 조건에서는 $o=00$ 이고, $i=1$ 인 뒤의 8 개 조건에서는 s 가 증가함에 따라 o 가 80, 40, 20, 10, 08, 04, 02, 01 순서로 이동한다.

그림 11 은 앞서 제시한 그림 7 의 파형을 보드 결과와 연결해 읽기 쉽도록 세로로 재배치한 것이다. 16 개 전수 시뮬레이션에서 핵심 조건은 $i=1$ 구간의 one hot 이동이다. 테스트벤치는 $\{i, s\}$ 를 0000 부터 1111 까지 순차 인가하고 실제 o 와 expected 를 비교하므로, 데이터 차단 상태와 여덟 선택 상태를 모두 검증한다.

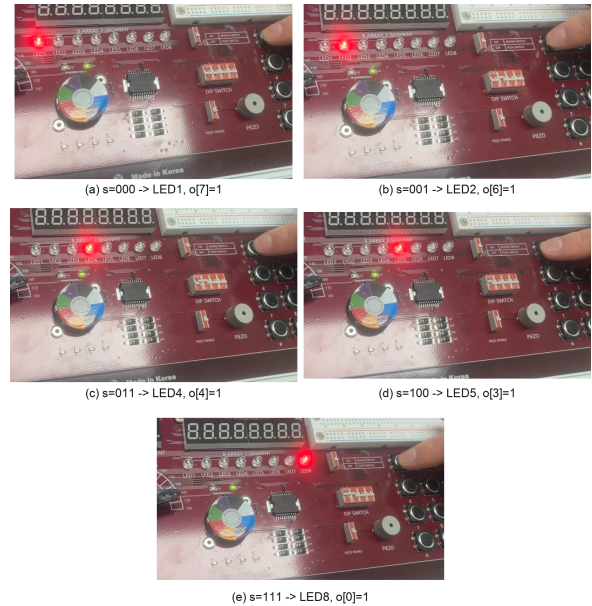


그림 12. Lab 07 보드 관찰. 키 1 로 $i=1$ 을 유지하고 DIP 스위치로 s 를 바꾸었을 때, 선택값에 대응하는 LED 하나만 점등되었다.

- $s=000$ 에서는 $o[7]=1$ 이 되어 LED1, $s=001$ 에서는 $o[6]=1$ 이 되어 LED2 가 점등되었다.
- $s=011$ 에서는 $o[4]=1$ 이 되어 LED4, $s=100$ 에서는 $o[3]=1$ 이 되어 LED5 가 점등되었다.
- $s=111$ 에서는 $o[0]=1$ 이 되어 LED8 이 점등되었고, 사용자 확인 내용과 동일했다.

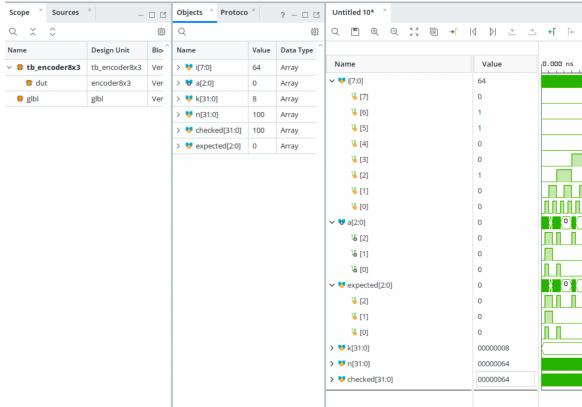
사진에 포함된 양 끝 선택값과 중간 선택값에서 두 개 이상의 LED 가 동시에 켜지지 않았다. 이는 RTL 의 one hot 성질뿐 아니라 $o[7]$ 부터 $o[0]$ 까지의 XDC 핀 배정이 보드의 LED1 부터 LED8 까지 올바르게 연결되었음을 보여준다. 촬영하지 않은 나머지 선택값은 Vivado 의 16 개 전수 파형으로 보완되며, 사진으로 확인한 다섯 조건은 모두 같은 순서와 일치하였다.

4.12.2 Lab 08 8 to 3 Encoder

Encoder 는 여덟 one hot 입력 중 어느 입력이 1 인지 3 비트 이진 코드로 줄이는 회로이다. 본 RTL 은 우선순위 인코더가 아니라 정확히 한 비트만 1 인 패턴을 case 문으로 판별한다. 따라서 i[7]부터 i[0]까지는 각각 000 부터 111 로 부호화되고, 입력이 없거나 둘 이상이 1 인 경우에는 default 값 000 을 출력한다.

Lab 08: signals and early intervals

Simulation - Behavioral Simulation - Functional - sim_1 - tb_encoder8x3



Lab 08: continued waveform and expected output

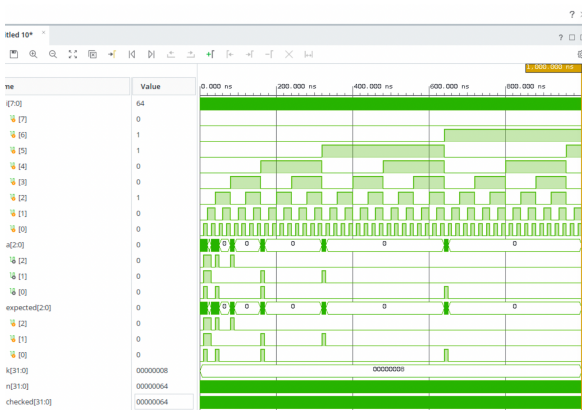
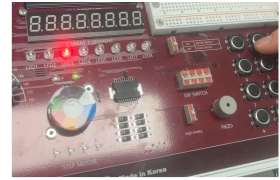


그림 13. Lab 08 Vivado 파형의 재구성. 테스트벤치가 256 개 입력 조합을 검사하며, 80, 40, 20, 10, 08, 04, 02, 01 의 유효 one hot 입력에서 a 가 0 부터 7 까지 변한다.

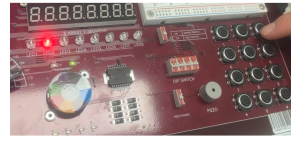
그림 13 은 앞서 제시한 그림 8 의 파형을 보드 결과와 연결해 읽기 쉽도록 세로로 재배치한 것이다. 유효 one hot 입력은 여덟 시점뿐이고, 나머지 248 개 입력은 default 분기로 처리된다. 따라서 checked=256 은 모든 조합을 검사했다는 의미이며, 보드 사진은 그중 실제 Encoder 동작을 정의하는 여덟 one hot 조건을 순서대로 확인한 기록이다.



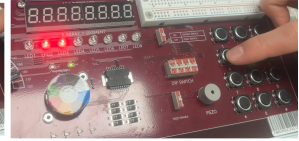
(a) Key 1, i[7]=1 -> a=000



(b) Key 2, i[6]=1 -> a=001



(c) Key 3, i[5]=1 -> a=010



(d) Key 4, i[4]=1 -> a=011



(e) Key 5, i[3]=1 -> a=100



(f) Key 6, i[2]=1 -> a=101



(g) Key 7, i[1]=1 -> a=110



(h) Key 8, i[0]=1 -> a=111

그림 14. Lab 08 보드 관찰. 키 1 부터 키 8 까지 하나씩 눌렀을 때 LED1, LED2, LED3 이 각각 a[2], a[1], a[0]을 나타내며 000 부터 111 까지의 이진 코드를 표시한다.

- 키 1~4: i[7], i[6], i[5], i[4]가 차례로 1 이 되며, a 는 000, 001, 010, 011 로 관찰되었다.
- 키 5~8: i[3], i[2], i[1], i[0]이 차례로 1 이 되며, a 는 100, 101, 110, 111 로 관찰되었다.

LED 패턴은 이진수의 자리값을 그대로 보여준다. 예를 들어 키 6 은 i[2]=1 이므로 a=101 이고 LED1 과 LED3 이 켜지며, 키 8 은 i[0]=1 이므로 a=111 이고 세 LED 가 모두 켜진다. 여덟 사진의 순차 변화가 RTL case 문의 매핑과 일치하므로, 논리식뿐 아니라 입력 키와 i[7:0], 출력 LED 와 a[2:0]의 실제 핀 연결도 함께 검증되었다.

4.12.3 시뮬레이션과 보드 결과의 대응

Vivado 시뮬레이션은 DEMUX 의 16 개 입력과 Encoder 의 256 개 입력을 자동 비교하여 RTL 의 기능을 검증한다. 보드 관찰은 같은 비트스트림에서 키와 DIP 스위치가 의도한 포트에 들어가고 LED 가 의도한 출력 비트를 표시하는지 검증한다. 두 단계의 결과가 모두 일치했으므로 별도의 RTL 수정이나 핀 재배정은 필요하지 않았다. 즉 시뮬레이션은 논리의 완전성을, 다중 조건의 사진은 물리 입출력과 비트 순서의 정확성을 보완적으로 입증한다.

5 고찰 및 토의

5.1 단계별 결과 비교

단계	검증 내용	근거
예상값	진리표·산술식·case 표로 기준 정의	전 Lab
VS Code	자기검사 TB 전체 입력 PASS	4~256 개 조건
Vivado	동일 RTL 과 TB 의 파형 비교	전 Lab 파형
FPGA 보드	핀 배정과 실제 LED 출력 확인	Lab 07-08

예상값, VS Code, Vivado 와 보드 결과는 서로 다른 오류를 검출한다. 자기검사 시뮬레이션은 논리 기능을 전수 확인하고, Vivado 파형은 같은 소스가 XSim 에서도 동일하게 동작하는지 보여준다. 보드 사진은 시뮬레이션이 확인하지 못하는 XDC 의 포트·핀 대응과 실제 출력 장치의 반응을 검증한다.

5.2 Vivado 실행 시간 문제

Lab 03 의 첫 Vivado 실행은 기본 실행 시간 1000 ns 에서 멈춰 256 개 중 100 개 조건만 표시되었다. 테스트벤치는 각 조건을 10 ns 씩 인가하므로 정상 종료에는 2560 ns 가 필요하다. run all 로 재실행한 뒤 n 과 checked 가 256 이고 마지막 F+F 에서 expected=1E 임을 확인하였다. 이 과정은 파형 창이 열렸다는 사실만으로 통과를 판단하지 않고 PASS 문구, 검사 수와 종료 시각을 함께 확인해야 함을 보여준다 [2, pp. 15, 35, 45].

5.3 자원 사용량과 경로 지연

모든 회로는 1~5 개의 Slice LUT 로 구현되었다. LUT 수가 논리식의 개수와 정확히 같지 않은 것은 FPGA LUT 가 여러 출력 또는 공통 입력을 가진 작은 함수를 묶어 구현할 수 있기 때문이다 [5]. 최장 입력-출력 경로는 8.603~12.330 ns 였고, 입력 수가 많은 가산기·감산기·비교기에서 상대적으로 길었다. 이 지연은 사람이 스위치나 키를 조작하는 시간보다 충분히 짧아 보드에서는 즉시 반응하는 것으로 관찰되었다.

5.4 경고와 제약

공통 경고는 클럭과 입출력 지연 제약이 없는 조합회로라는 점에서 발생했다. Timing 38-313 과 Place 46-29 는 타이밍 목표가 없음을, Power 33-232 는 클럭 활동 정보가 없음을 나타낸다. CFGBVS-1 은 구성 전압 속성 미지정 경고이다. 이번 실험의 기능 검증과 비트스트림 생성은 완료되었지만, 이후

순차회로에서는 create_clock 와 입출력 지연을 포함한 타이밍 제약을 추가해야 한다 [3].

5.5 회로별 특징

- Encoder 의 $i[7]=1$ 출력 000 은 입력이 없는 상태의 000 과 구분되지 않는다. 실제 시스템에서는 valid 신호가 필요하다.
- DEMUX 는 $s=000$ 에서 $o[7]$, Decoder 는 $abc=000$ 에서 $o[0]$ 을 선택한다. 두 회로의 비트 방향은 RTL 에서 확인해야 한다.
- 7 Segment 는 10~15 도 표시하므로 BCD 보다 16 진수 한 자리 디코더라는 표현이 정확하다.

5.6 보드 기록 환경

비트스트림은 Apple Silicon macOS 의 Vivado 2026.1 환경에서 생성하고, 실험 당시 보드 기록은 Windows PC 의 Vivado Hardware Manager 를 사용했다. Xilinx Platform Cable USB II 를 macOS 컨테이너에서 직접 사용하기 어려워 비트스트림을 옮겨 기록했으며, 동일 소자용 비트스트림이므로 기록 PC 가 달라도 구현 회로는 변하지 않는다. 이후에는 USB/IP 를 통해 컨테이너의 hw_server 가 케이블을 인식하는 단계까지 확인하였다.

6 결론

10 종의 조합논리회로를 Verilog 로 구현하고 VS Code 자기검사와 Vivado Behavioral Simulation 으로 검증하였다. 모든 프로젝트에서 합성·구현과 비트스트림 생성이 완료되었으며 Slice LUT 사용량은 1~5 개, 최장 입력-출력 지연은 8.603~12.330 ns 였다. 조교 선정 실험인 Lab 07 DEMUX 에서는 다섯 선택값에서 한 LED 만 이동하는 one hot 분배를 확인했고, Lab 08 Encoder 에서는 키 1~8 의 모든 유효 one hot 입력이 000~111 로 부호화되는 과정을 확인했다. 두 실험의 다중 보드 사진은 Vivado 전수 파형, RTL 의 비트 순서와 XDC 핀 대응에 모두 일치하였다. 첫 실행에서 Lab 03 파형이 1000 ns 에서 중단된 문제도 실행 시간을 2560 ns 이상으로 늘려 전수 검증하였다. 결과적으로 조합논리 기능, 구현 가능성, 실제 장치 동작을 순차적으로 검증하였다.

7 참고문헌

- [1] 이해리, 『전자전기컴퓨터설계실험 II LAB 0 오리엔테이션』, 서울시립대학교, pp. 32, 34-37, 2026.

- [2] 이해리, 『전자전기 컴퓨터 설계 실험 II LAB 1 조합회로 실습』,
04.LAB1_00_CONTENTS p. 10 및
04.LAB1_03_ADDER4_VIVADO pp. 15, 35, 45, 50, 2026.
- [3] AMD, Vivado Design Suite User Guide Using Constraints,
UG903, v2026.1, 2026-07-01, 확인일 2026-09-20.
<https://docs.amd.com/r/en-US/ug903-vivado-using-constraints/Project-Flows>
- [4] AMD, Vivado Design Suite User Guide Implementation, UG904,
v2026.1, 2026-06-23, 확인일 2026-09-20.
<https://docs.amd.com/r/en-US/ug904-vivado-implementation/Implementation-Sub-Processes>
- [5] AMD, 7 Series FPGAs Configurable Logic Block User Guide,
UG474, Rev. 1.9, 2025-04-01, 확인일 2026-09-20.
https://docs.amd.com/r/en-US/ug474_7Series_CLB/CLB-Overview